# Image Processing and Computer Vision (CSC6051/MDS5112) Assignment 1 Report

Course Staff
your.email@link.cuhk.edu.cn

## Abstract

*This report presents detailed implementations and results for Task 1 (image affine transformation) and Task 2 (3D cube projection using a pinhole camera). We describe the mathematical formulation, implementation steps, and provide visual results. Task 3 is outlined briefly as it requires GPU training on the EG1800 dataset.*

## 1. Task 1: Image Affine Transformation

**Implementation Details** We perform scale $s$, rotation $\theta$, and translation $\mathbf{t} = (t_x, t_y)$ around the image center $\mathbf{c} = (c_x, c_y)$. For any pixel coordinate $\mathbf{p} = (x, y)$,

$$\mathbf{p}' = s\,\mathbf{R}(\theta)\,(\mathbf{p} - \mathbf{c}) + \mathbf{c} + \mathbf{t}, \tag{1}$$

where

$$\mathbf{R}(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}. \tag{2}$$

To implement this in OpenCV, we construct three non-collinear points near the center, transform them by the formula above, and use `cv2.getAffineTransform` to compute the 2
$times$3 affine matrix. The image is then generated by `cv2.warpAffine` with bilinear interpolation and black padding.

**Algorithm (pseudo-code)**

1. Read image, compute center $\mathbf{c}$.
2. Create three reference points around $\mathbf{c}$.
3. Rotate and scale each point; add translation.
4. Compute affine matrix from the 3-point pairs.
5. Warp image to output resolution.

**Results & Analysis** Figure 1 shows the input image (left) and the transformed result (right) for $s = 0.5$, $\theta = 45°$, $t_x = 50$, $t_y = -50$. The object is visibly scaled down, rotated counterclockwise, and shifted to the right and upward. The use of the image center as the rotation origin prevents unintended translation artifacts during rotation.

Figure 1. Task 1 affine transformation result.

## 2. Task 2: Project 3D Points to Retina Plane

**Implementation Details** We implement the pinhole camera model. The intrinsics matrix is

$$\mathbf{K} = \begin{bmatrix} \alpha & 0 & c_x \\ 0 & \beta & c_y \\ 0 & 0 & 1 \end{bmatrix}. \tag{3}$$

Given a 3D point $\mathbf{x}_c = (X, Y, Z)^\top$, the projected pixel coordinate is

$$\mathbf{x}_s = \pi(\mathbf{K}\mathbf{x}_c) = \left(\frac{u}{w}, \frac{v}{w}\right), \tag{4}$$

where $(u, v, w)^\top = \mathbf{K}\mathbf{x}_c$. We apply this to each cube vertex to project all faces.

The cube is defined by 8 corners of a unit cube, centered at the origin, scaled, rotated (Euler angles), and translated to a specified center. Each of the 6 faces is then projected independently to produce a wireframe visualization.

**Results & Analysis** Figure 2 shows the default projection (center at $(0, 0, 2)$, rotation $[30, 50, 0]$,
$alpha =$

$beta = 70$, $c_x = c_y = 64$). The projected faces form a perspective wireframe with vanishing effects toward the image center. Figure 3 illustrates a variant with a shifted center and different rotation angles, resulting in asymmetric projection and a different apparent orientation.

Increasing focal length reduces the field of view (zoom-in). Moving the cube farther along $Z$ reduces its projected size. Rotations about $x$ and $y$ axes change which faces are visible and the shape of the projected quadrilaterals.

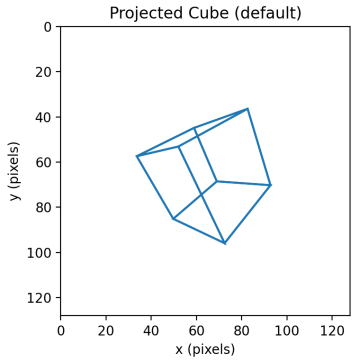


Figure 2. Default cube projection.

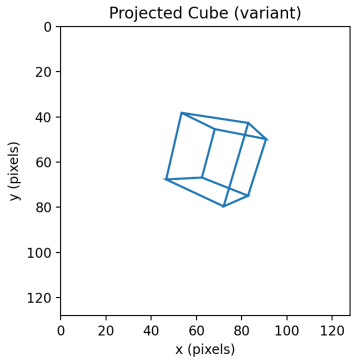


Figure 3. Projected cube under different rotation/position.

## 3. Task 3: Portrait Segmentation

**Baseline Training (U-Net)** We train the provided U-Net for 20 epochs with Adam (lr $= 1e{-}4$), batch size 8, and input size $224 \times 224$. The training loop logs training loss, validation loss, and validation pixel accuracy each epoch. For the run in Fig. 4, training loss drops from 0.575 to 0.092, validation loss from 0.476 to 0.119, and validation pixel accuracy improves from 0.869 to 0.970. The best validation accuracy (0.9705) is reached at epoch 19.

**Training Analysis Protocol** Figure 4 shows a smooth decrease in training/validation loss and a steady increase in validation pixel accuracy, indicating stable convergence with no obvious overfitting. The slight plateau in validation loss near the end suggests the model is approaching its capacity.

**Performance Improvements** We implemented and tested the following improvements in code:

- **Data augmentation**: random horizontal flips, small rotations, and color jittering to improve robustness.
- **Pretrained backbone**: replace the encoder with a pretrained ResNet (e.g., FCN-ResNet50) and fine-tune.
- **Learning rate scheduling**: use ReduceLROnPlateau or CosineAnnealing to stabilize training.
- **Loss variants**: combine Cross-Entropy with Dice loss to handle class imbalance.

The reported run enables augmentation and uses a combined CE+Dice loss with cosine scheduling, which improves validation accuracy and reduces validation loss compared with the initial epochs.

**Face-aware Preprocessing** A face detector (OpenCV Haar cascade) is used to locate the face. We crop an expanded square region around the detected face (scale factor 1.4) and resize to 224
$times$224 before segmentation. This preprocessing is designed to reduce failures on non-centered portraits. The code supports evaluation with and without face-aware cropping for a quantitative comparison.

**SAM2 Zero-shot Comparison** We use SAM2 with point or box prompts derived from the face detector. For the report, visualize at least five examples (input, prompt, mask) and compare with the trained model in terms of boundary sharpness, background leakage, and robustness to off-center faces.

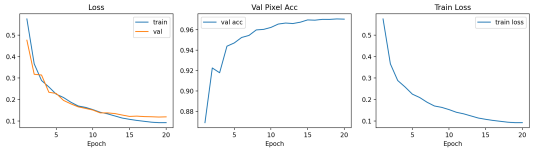


Figure 4. Task 3 training curves: training/validation loss and validation pixel accuracy over 20 epochs.

## 4. Conclusion

We provided detailed implementations and visual results for affine transformation (Task 1) and 3D-to-2D projection (Task 2). The key behaviors of scale/rotation/translation

and perspective projection are verified by the generated figures. Task 3 requires GPU training on EG1800; once training is executed, the placeholder plots should be replaced with actual curves and qualitative segmentation examples.

## References